

Entorno de Simulación Robótica con Drake

Sebastian Allende

Resumen—Este documento describe la configuración y validación de un entorno de simulación robótica basado en Drake para el brazo Franka Panda, junto con el proceso completo de instalación del framework desde código fuente mediante Bazel. Se presenta una arquitectura de tres procesos independientes—generación de trayectoria, control por dinámica inversa y simulación física— que se comunican exclusivamente a través de canales LCM (Lightweight Communications and Marshalling). Este diseño modular permite modificar, depurar o reemplazar cualquiera de los tres componentes de forma independiente, incluyendo la eventual sustitución del proceso de simulación por hardware real sin alterar el resto del sistema. Se detallan los requisitos de hardware y software, los pasos de compilación e instalación de Drake, y la implementación de cada proceso, sentando así una base reutilizable para futuras investigaciones en control robótico.

I. INTRODUCCIÓN

La simulación realista de manipuladores robóticos es una herramienta clave para el desarrollo y validación de estrategias de control antes de su despliegue en hardware real. Drake, desarrollado por el Massachusetts Institute of Technology (MIT) y Toyota Research Institute (TRI), es un framework de modelado y simulación orientado específicamente a robótica y control, que combina dinámica multicuerpo precisa, detección de colisiones y herramientas de optimización en un mismo entorno.

Este trabajo documenta la puesta en marcha de un entorno de simulación para el brazo robótico Franka Panda utilizando Drake, con un enfoque particular en la arquitectura de comunicación entre procesos. En lugar de implementar el sistema completo (generación de trayectoria, control y simulación física) como un único programa integrado, se optó por dividirlo en tres procesos independientes que se sincronizan únicamente a través de canales LCM. Esta decisión de diseño busca replicar, en la medida de lo posible, la arquitectura típica de un sistema robótico real, donde percepción, planificación y control suelen correr en procesos o incluso máquinas separadas.

El documento se organiza de la siguiente manera: la Sección II detalla los requisitos de sistema y el proceso de instalación de Drake desde código fuente, tanto para uso en C++ como en Python. La Sección III describe la arquitectura de la simulación robótica implementada, incluyendo el detalle de cada uno de los tres procesos y los canales LCM que los conectan. Finalmente, se presentan las conclusiones y líneas de trabajo futuro.

II. SETUP

Requisitos del Sistema

Hardware:

- **RAM:** mínimo 8 GB, recomendado 16 GB. La compilación de Drake desde el código fuente con Bazel es intensiva en memoria; con menos de 8 GB es necesario limitar el paralelismo (`--jobs=1`) y el proceso puede tardar varias horas.
- **Almacenamiento:** al menos 30 GB libres (el repositorio de Drake y la caché de Bazel ocupan un espacio considerable).

Software base: El entorno fue configurado y probado con las siguientes versiones:

- **Sistema operativo:** Ubuntu 24.04.3 LTS (*Noble Numbat*)
- **Python:** 3.12.3
- **Bazel** (gestionado por Bazelisk): 9.1.1

Dependencias del Sistema

Antes de compilar Drake deben instalarse los siguientes paquetes:

```
sudo apt update

sudo apt install -y \
  build-essential \
  cmake \
  git \
  curl \
  python3 \
  python3-pip \
  python3-venv \
  gcc \
  g++ \
  unzip \
  zip
```

Instalación de Bazelisk

Drake utiliza **Bazel** como sistema de compilación. En lugar de instalar Bazel directamente se recomienda usar **Bazelisk**, que descarga y gestiona automáticamente la versión correcta de Bazel que requiere el repositorio (especificada en el archivo `.bazelversion`).

```
wget https://github.com/bazelbuild/bazelisk/releases/latest/download/bazelisk-linux-amd64

chmod +x bazelisk-linux-amd64

sudo mv bazelisk-linux-amd64 /usr/local/bin/bazelisk
```

Verificar la instalación:

```
bazelisk --version
```

Repositorio oficial: <https://github.com/bazelbuild/bazelisk>

Clonación del Repositorio de Drake

```
git clone https://github.com/RobotLocomotion/drake.git

cd drake
```

Repositorio oficial: <https://github.com/RobotLocomotion/drake>

Compilación e Instalación

Compilación: Desde el directorio raíz del repositorio clonado:

```
cd ~/drake

bazelisk build //... --jobs=1
```

La opción `--jobs=1` limita el paralelismo para reducir el consumo de memoria durante la compilación. El proceso puede tardar varias horas dependiendo del hardware disponible.

Instalación en directorio local: Una vez compilado, Drake se instala en un directorio local (`~/drake-install`) para poder usarlo fuera del entorno de Bazel:

```
bazelisk run //:install -- ~/drake-install
```

Esto genera la estructura:

```
~/drake-install/
bin/
include/
lib/
  libdrake.so
  python3.12/
    site-packages/
      pydrake/
share/
```

Uso en C++

Los proyectos propios que usen Drake con C++ se ubican en un repositorio separado, independiente del repositorio de Drake. La compilación ya no depende de Bazel: se utiliza CMake, apuntando a la instalación local de Drake generada en la Sección anterior (`~/drake-install`).

El archivo `CMakeLists.txt` en la raíz del proyecto declara la dependencia hacia Drake y agrega los subdirectorios correspondientes a cada módulo:

```
cmake_minimum_required(VERSION 3.16)
project(NombreProyecto)
set(CMAKE_CXX_STANDARD 17)

# Apunta a la instalacion local de Drake
find_package(drake CONFIG REQUIRED
  PATHS $ENV{HOME}/drake-install/lib/cmake/
  drake
)

add_subdirectory(modulo_1)
add_subdirectory(modulo_2)
# add_subdirectory(modulo_n) # descomenta
# segun se necesite
```

Cada subdirectorio contiene su propio `CMakeLists.txt`, donde se declaran los targets específicos (ejecutables o librerías) y sus dependencias de Drake.

Para compilar el proyecto completo:

```
cd ~/ruta_del_proyecto
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

Una vez compilado, cada target genera un ejecutable dentro del directorio `build/`, que puede correrse directamente sin depender de Bazel ni del entorno de Drake:

```
./build/<ruta_al_ejecutable>
```

A diferencia del flujo con Bazel, este comando no recompila automáticamente si hay cambios en el código; es necesario volver a ejecutar `make` (o `cmake --build .` desde el directorio `build/`) antes de correr el ejecutable nuevamente.

Uso en Python

Para usar `pydrake` desde scripts Python fuera de Bazel, se debe agregar la ruta de instalación al `PYTHONPATH`.

Configuración del `PYTHONPATH`: Las siguientes líneas fueron agregadas al archivo `~/bashrc`:

```
export DRAKE_RESOURCE_ROOT=$HOME/drake-install
/share
export PYTHONPATH=$HOME/drake-install/lib/
python3.12/site-packages:$PYTHONPATH
```

Para aplicar los cambios en la terminal actual:

```
source ~/.bashrc
```

Las terminales nuevas cargarán estas variables automáticamente.

Verificación: Para verificar la correcta instalación y funcionamiento se puede ejecutar el siguiente código:

```
import pydrake
from pydrake.systems.framework import
    DiagramBuilder
from pydrake.systems.analysis import Simulator

print("pydrake_OK")
```

III. SIMULACIÓN ROBÓTICA

Descripción General

El proyecto implementa una simulación de un brazo robótico **Franka Panda** en Drake con control basado en dinámica inversa. El sistema se divide en **tres procesos independientes** que se comunican entre sí a través de **LCM** (*Lightweight Communications and Marshalling*):

1. **Traectoria** — genera la referencia de movimiento deseado.
2. **Controlador** — calcula los torques de control.
3. **Simulación** — ejecuta la simulación física y la visualización.

Cuadro I
CANALES LCM DEL SISTEMA.

Canal	Tipo LCM	Contenido	Sentido
DESIRED_STATE	lcmt_drake_signal	$[q_d; \dot{q}_d]$ (18 val.)	trayectoria → controlador
PANDA_COMMAND	lcmt_panda_command	τ (9 val.)	controlador → simulación
PANDA_STATUS	lcmt_panda_status	$[q; \dot{q}]$ (18 val.)	simulación → controlador

Los tres procesos corren en paralelo y se sincronizan únicamente a través de los canales LCM. La Fig. 1 muestra la arquitectura general y la Fig. 2 detalla la estructura interna del controlador.

Diagrama de Bloques

La tabla I resume los canales LCM utilizados.

Proceso 1: Generador de Trayectoria (*trayectoria.cc*)

Es el proceso más simple: genera un estado deseado $[q_d; \dot{q}_d]$ que varía en el tiempo y lo publica en el canal DESIRED_STATE.

A continuación se detalla como se definió en este caso la trayectoria usada de prueba, pero esta puede ser reemplazada por cualquier otra trayectoria respetando el protocolo de comunicación.

SetpointSinusoidal: Clase `LeafSystem` que calcula el setpoint en cada paso de tiempo. El **joint 0** (rotación de la base) oscila sinusoidalmente alrededor de la posición *home*:

$$q_0(t) = q_0^{\text{home}} + A \sin(\omega t), \quad \dot{q}_0(t) = A \omega \cos(\omega t)$$

con amplitud $A = 0,4$ rad y frecuencia $f = 0,1$ Hz (período de 10 s). Los **8 joints restantes** permanecen fijos en la posición *home*:

$$q^{\text{home}} = [0, -0,5, 0, -2, 0, 1,5, 0, 0,04, 0,04]$$

VectorToLcmSignal: Convierte el vector de 18 valores en un mensaje `lcmt_drake_signal` para ser publicado por LCM. El canal se publica con un período de 5 ms (200 Hz).

Proceso 2: Controlador (*franka_task_control.cc*)

Lee el estado actual del robot y la referencia deseada, y calcula los torques necesarios para seguir la trayectoria usando control por dinámica inversa.

Entradas LCM:

- PANDA_STATUS → estado medido $[q; \dot{q}]$.
- DESIRED_STATE → estado deseado $[q_d; \dot{q}_d]$. Mientras no llega el primer mensaje, usa la posición *home* como valor por defecto (clase `DesiredStateFromLcm`).

ControladorPanda y la Planta Espejo:

`ControladorPanda` es un `Diagram` que encapsula el `InverseDynamicsController` de Drake (Fig. 2).

El controlador por dinámica inversa requiere un modelo interno del robot para calcular los torques. Este modelo, llamado **planta espejo** (ya que copia a la planta de la simulación principal), es una instancia separada de `MultibodyPlant` cargada desde el mismo URDF del Franka Panda, pero sin entorno (sin cubo, sin suelo, sin geometrías de colisión). Su única función es proveer la información de masa, inercia y cinemática para el cálculo de torques.

Los parámetros del controlador son:

$$k_p = 1500, \quad k_i = 0, \quad k_d = 150 \quad (\text{por joint})$$

Salida LCM: Los torques calculados se empaquetan en `lcmt_panda_command` y se publican en PANDA_COMMAND con un período de 5 ms.

Proceso 3: Simulación (*main.cc* + *PlantaComun*)

Ejecuta la simulación física del robot y el entorno, y publica el estado resultante.

PlantaComun: Clase `Diagram` definida en `planta_comun.h` / `franka_simulation.cc` que encapsula todos los bloques de la simulación:

- **MultibodyPlant** (paso de integración: 1 ms):
 - Brazo Franka Panda (`panda_arm_hand.urdf`), soldado al mundo.
 - Cubo rígido de $0,1 \times 0,1 \times 0,1$ m, masa 0.1 kg, fricción estático/dinámico: 2.0/1.5.
 - Suelo plano con fricción 1.0/1.0.
 - Gravedad: $g = 9,81 \text{ m/s}^2$ en $-z$.
- **SceneGraph**: geometrías de colisión y visualización.
- **LcmSubscriberSystem** (PANDA_COMMAND) → **PandaCommandReceiver**: aplica los torques a los actuadores.
- **PandaStatusSender** → **LcmPublisherSystem** (PANDA_STATUS): publica $[q; \dot{q}]$ cada 5 ms.
- **Demultiplexer**: separa $[q; \dot{q}]$ en posición y velocidad.

Visualización con Meshcat: `main.cc` agrega un `MeshcatVisualizer` conectado al puerto de consulta de geometría de `PlantaComun`. Al iniciar la simulación se imprime la URL local (`http://localhost:7000`). La posición inicial del cubo se establece en $x = 0,2$ m, $y = 0$ m, $z = 0,035$ m. La simulación corre en tiempo real (`target_realtime_rate = 1.0`) y avanza indefinidamente.

Ejecución

Los tres procesos se ejecutan en terminales separadas, en cualquier orden:

```
# Terminal 1
bazelisk run //ruta al archivo:main

# Terminal 2
bazelisk run //ruta al archivo:
    franka_task_control

# Terminal 3
bazelisk run //ruta al archivo:trayectoria
```

Una vez corriendo los tres, abrir la URL de Meshcat en el navegador para ver la simulación en tiempo real.

Se observa en las Fig. 3 y 4 una referencia del entorno de simulación físico del robot con el cubo a su lado.

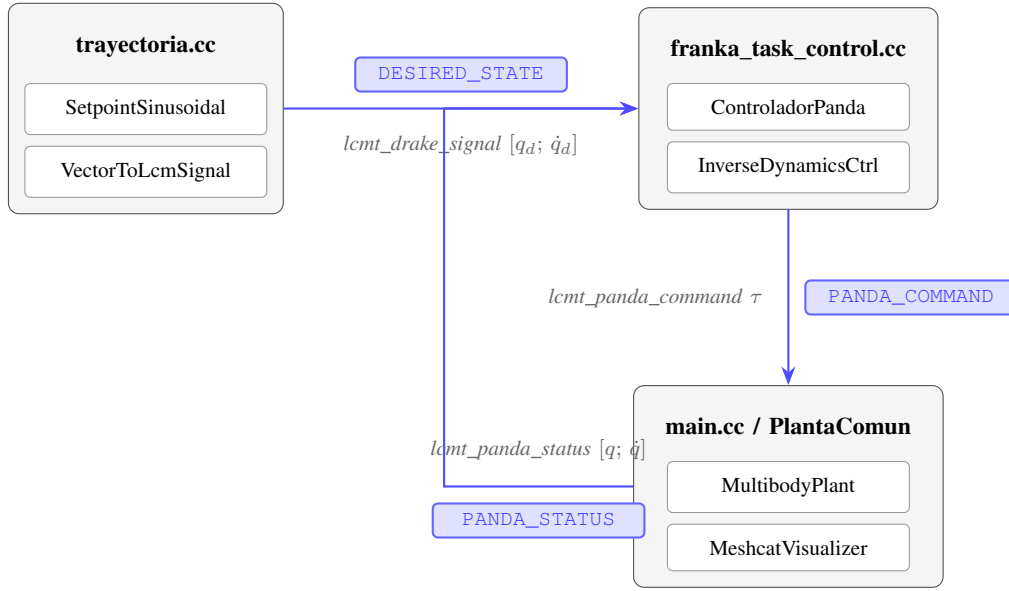


Figura 1. Arquitectura de comunicación entre los tres procesos vía LCM.

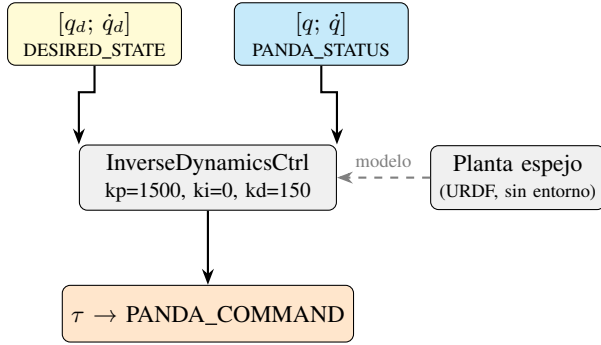


Figura 2. Estructura interna del `ControladorPanda`.

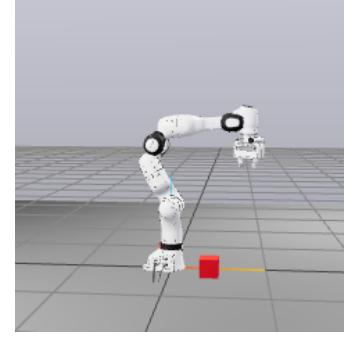


Figura 4.

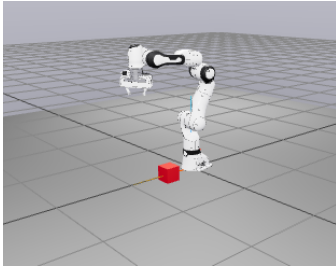


Figura 3.

IV. CONCLUSIONES

Se implementó exitosamente un entorno de simulación para el brazo robótico Franka Panda sobre Drake, estructurado en tres procesos independientes –trayectoria, controlador y simulación– comunicados mediante canales LCM. Esta arquitectura modular cumple el objetivo de permitir modificar, depurar o sustituir cualquiera de los tres componentes sin necesidad de alterar los otros dos, lo cual quedó validado durante el

desarrollo al iterar sobre el controlador de dinámica inversa sin requerir cambios en los procesos de trayectoria o simulación.

Más allá de su utilidad inmediata, este diseño tiene una ventaja estratégica: al depender únicamente de mensajes LCM y no de llamadas directas entre procesos, el proceso de simulación física podría reemplazarse en el futuro por una interfaz con el hardware real del Franka Panda, reutilizando íntegramente los procesos de trayectoria y control ya desarrollados. Esto convierte al entorno actual en una plataforma de transición natural entre simulación y experimentación física.

Como trabajo futuro se identifican las siguientes líneas:

- Incorporar métricas cuantitativas de desempeño (error de seguimiento de trayectoria, latencia de los canales LCM) para validar formalmente el controlador.
- Extender el controlador por dinámica inversa a estrategias más avanzadas aprovechando la modularidad del sistema.
- Migrar el proceso de simulación a una interfaz de hardware real, validando la robustez de la arquitectura LCM fuera de un entorno puramente simulado.

Este entorno queda establecido como base para futuras

investigaciones en control robótico.

V. REFERENCIAS

- Documentación oficial de Drake: <https://drake.mit.edu>
- Repositorio de Drake: <https://github.com/RobotLocomotion/drake>
- Bazelisk: <https://github.com/bazelbuild/bazelisk>
- Sistema de compilación Bazel: <https://bazel.build>